

Instructions

- Homework 5 was due Wednesday, August 26th at 18:00 Chicago Time.
 - All numbers and figures come from `homework_5_solution.ipynb`, run from top to bottom.
-

Problem 1: PCA from Scratch

In this problem, you will implement Principal Component Analysis (PCA) “by hand” (using basic linear algebra functions) and compare it to the standard implementation.

Generate a dataset with $n = 500$ observations and 3 highly correlated features.

$$\mu = [0, 0, 0]$$

$$\Sigma = \begin{pmatrix} 1 & 0.9 & 0.7 \\ 0.9 & 1 & 0.8 \\ 0.7 & 0.8 & 1 \end{pmatrix}$$

```
1 import numpy as np
2 np.random.seed(42)
3 mean = [0, 0, 0]
4 cov = [[1, 0.9, 0.7], [0.9, 1, 0.8], [0.7, 0.8, 1]]
5 X = np.random.multivariate_normal(mean, cov, 500)
```

Problem 1.1: Eigendecomposition

- Calculate the covariance matrix of the data X manually (recall $\text{Cov}(X) = \frac{1}{n-1}X^T X$ if X is centered).
- Find the eigenvalues and eigenvectors of this covariance matrix using `np.linalg.eig`.
- Report the eigenvalues and eigenvectors.

Answer:

The covariance matrix is symmetric, so use `np.linalg.eigh`: it returns real eigenvalues in ascending order. `np.linalg.eig` is the general routine, and here it returns complex dtype in no particular order.

```
1 X_demeaned = X - X.mean(axis=0)
2 covX = 1 / 499 * X_demeaned.T @ X_demeaned
3
4 eigenvalues, eigenvectors = np.linalg.eigh(covX)
5 order = np.argsort(eigenvalues)[::-1]
6 eigenvalues, eigenvectors = eigenvalues[order], eigenvectors[:, order]
```

Sorted largest first:

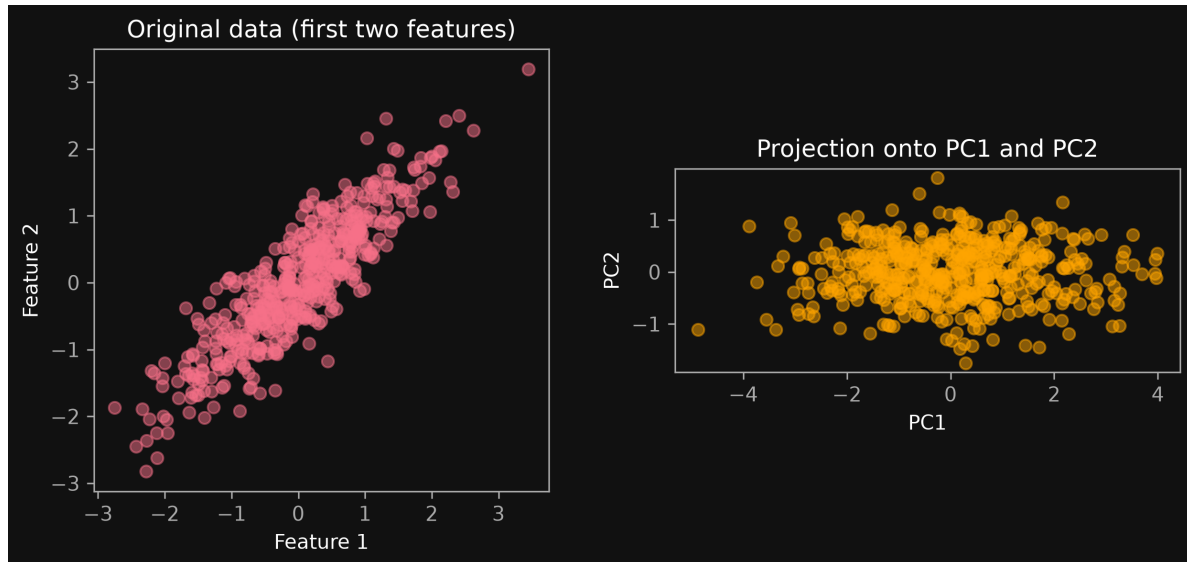
$$\lambda = (2.290778, 0.313006, 0.087826), \quad Q = \begin{bmatrix} -0.585990 & -0.571257 & -0.574701 \\ -0.604994 & -0.163388 & 0.779286 \\ -0.539071 & 0.804345 & -0.249863 \end{bmatrix}$$

where the columns of Q are the eigenvectors in the same order. The three features are strongly correlated, so PC1 alone carries 85.1% of the variance, against 11.6% and 3.3% for the other two.

Problem 1.2: Projection

- Project the data X onto the principal components (the eigenvectors).
- Plot the original data (pick any 2 dimensions) and the projected data (PC1 vs PC2).

Answer:



Both panels are drawn on equal axes. The raw features lie on a tilted cigar, because they are correlated. Rotating onto the eigenvectors removes the tilt: PC1 and PC2 are uncorrelated by construction, and almost all of the spread has been moved onto PC1.

Problem 1.3: Verification

- Use `sklearn.decomposition.PCA` to perform PCA on the same data.
- Compare the components (eigenvectors) and explained variance (eigenvalues) from `sklearn` with your manual calculation. Are they the same?

Answer:

Yes. The eigenvalues agree with `pca.explained_variance_` to 4.4×10^{-16} , and the eigenvectors agree with `pca.components_` up to a sign flip on PC1.

The sign is arbitrary: if v is a unit eigenvector then so is $-v$, and nothing in the decomposition prefers one. `sklearn` pins it down by a convention on the loadings; `eigh` does not.

Problem 2: The Pitfalls of Principal Component Regression (PCR)

A common misconception is that the principal components with the largest variance are always the most useful features for prediction. This problem illustrates a scenario where this is **not** true.

Generate the following dataset:

```
1 import numpy as np
2 import pandas as pd
3 import statsmodels.api as sm
4 from sklearn.decomposition import PCA
5
6 np.random.seed(42)
7 n = 500
8
9 X_high_var = np.random.normal(0, 10, (n, 7))
10 X_low_var = np.random.normal(0, 1, (n, 3))
11 X = np.hstack([X_high_var, X_low_var])
12
13 true_beta = np.array([0]*7 + [2, -2, 2])
14 y = X @ true_beta + np.random.normal(0, 0.5, n)
```

Problem 2.1: PCA Analysis

Run PCA on the raw input matrix X . You must center the data first ($\tilde{X} = X - \bar{X}$).

- Report the explained variance ratio for all 10 components.
- Which components capture the majority of the variance?

Answer:

	explained variance ratio
PC1	0.16773
PC2	0.16159
PC3	0.15787
PC4	0.14255
PC5	0.12969
PC6	0.12647
PC7	0.10983
PC8	0.00149
PC9	0.00142
PC10	0.00136

PC1 to PC7 hold 99.57% of the variance, PC8 to PC10 the remaining 0.43%. The split is mechanical: the first seven columns of X were drawn with $\sigma = 10$ and the last three with $\sigma = 1$, a variance ratio of 100 to 1.

Problem 2.2: PCR - Regressing on High Variance Components

Fit an OLS model using **only** the first 7 principal components to predict y .

- Report the R^2 of this model.

- Does capturing the majority of the variance ensure good predictive power?

Answer:

$$R^2 = 0.0064.$$

No. Those seven components hold 99.57% of the variance in X and explain 0.6% of y . PCA never looks at y , so ranking components by variance ranks them by how much they move, not by how much they matter.

Problem 2.3: PCR - Regressing on Low Variance Components

Fit an OLS model using **only** the last 3 principal components (PC8, PC9, PC10) to predict y .

- Report the R^2 of this model.

Answer:

$R^2 = 0.9750$, from the three components that carry 0.43% of the variance.

Problem 2.4: Conclusion

Based on your results, explain why simply selecting the top k principal components for a regression model might be dangerous.

Answer:

Because variance in X and predictive power for y are unrelated quantities, and here they are exactly reversed: the top seven components give $R^2 = 0.0064$ and the bottom three give $R^2 = 0.9750$. Truncating at the top k would have thrown away the entire signal while keeping all of the noise.

Variance also depends on units. Rescaling one column in meters rather than kilometers changes its variance by 10^6 and reorders the components, without changing the regression at all.

If the goal is prediction, select on something that sees y : cross-validated Lasso, partial least squares, or cross-validating over which components to keep rather than assuming the top ones.

Problem 3: Bagging from Scratch

In class, we discussed how Bagging (Bootstrap Aggregating) reduces variance by averaging predictions from multiple models trained on bootstrapped samples. You will implement this manually.

Generate a synthetic “checkerboard” dataset:

```
1 import pandas as pd
2 np.random.seed(42)
3 n_points = 500
4 grid_size = 4
5 limit = 10
6 step = (limit * 2) / grid_size
7
8 all_points = []
9 all_labels = []
```

```

10
11 for i in range(grid_size):
12     for j in range(grid_size):
13         label = (i + j) % 2
14         x_min, x_max = -limit + i * step, -limit + (i + 1) * step
15         y_min, y_max = -limit + j * step, -limit + (j + 1) * step
16         points_x = np.random.uniform(x_min, x_max, int(n_points/16))
17         points_y = np.random.uniform(y_min, y_max, int(n_points/16))
18         all_points.append(np.vstack((points_x, points_y)).T)
19         all_labels.extend([label] * int(n_points/16))
20
21 X = np.vstack(all_points)
22 y = np.array(all_labels)

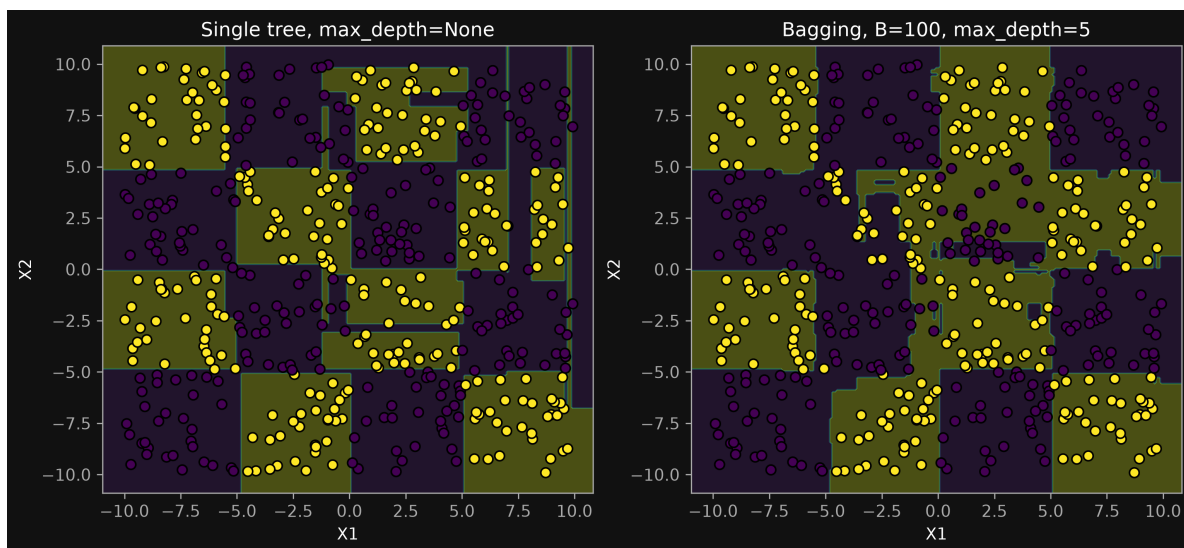
```

Problem 3.1: Single Decision Tree

Fit a single `DecisionTreeClassifier` (from `sklearn`) with `max_depth=None` (or a high number like 10) to the data.

- Visualize the decision boundary (you can use the `meshgrid` method shown in class).
- Does the decision boundary look smooth? Does it look like it's overfitting?

Answer:



Not smooth, and yes it is overfitting. The boundary is a staircase of axis-aligned rectangles, and several are thin slivers wrapped around one or two points, well inside a square where every point carries the same label. Training accuracy is 1.0000: with `max_depth=None` the tree splits until every leaf is pure, so it reproduces the noise as well as the checkerboard.

Problem 3.2: Manual Bagging Implementation

Implement Bagging manually (do **not** use `BaggingClassifier`).

- Create a loop that runs $B = 100$ times.
- Inside the loop:
 1. Create a bootstrap sample of the dataset (sample N rows with replacement).
 2. Fit a new `DecisionTreeClassifier` (`max_depth=5`) on this bootstrap sample.
 3. Store the trained tree in a list.
- To predict for a new point (or the meshgrid for plotting), get predictions from all 100 trees and take the majority vote (average).

Visualize the decision boundary of your manual Bagging ensemble.

Answer:

The ensemble boundary is the right-hand panel of the figure above.

```

1 class BaggingDTC:
2     def __init__(self, B=100, max_depth=5, seed=0):
3         self.B = B
4         self.max_depth = max_depth
5         self.rng = np.random.RandomState(seed)
6         self.trees = []
7
8     def fit(self, X, y):
9         n_samples = X.shape[0]
10        for _ in range(self.B):
11            idx = self.rng.choice(n_samples, size=n_samples, replace=True)
12            tree = DecisionTreeClassifier(max_depth=self.max_depth, random_state
=42)
13            self.trees.append(tree.fit(X[idx], y[idx]))
14        return self
15
16    def predict(self, X):
17        votes = np.array([tree.predict(X) for tree in self.trees])
18        return (votes.mean(axis=0) >= 0.5).astype(int)

```

Problem 3.3: Comparison and Interpretation

Compare the decision boundary of your Bagging ensemble to the single decision tree in 3.1. Is it smoother? Relate what you see to variance reduction.

Answer:

Yes. The slivers are gone and the boundary sits close to the true grid lines at ± 5 and 0.

Each bootstrap sample omits about 37% of the data, so each tree puts its spurious splits in a different place. Those splits are variance, not signal, and they do not survive a vote across 100 trees, while the real boundaries appear in nearly every tree and do. Averaging B predictors reduces the variance of the average without changing its expectation, which is the whole mechanism.

Training accuracy falls from 1.0000 to 0.9194. That is the trade: the ensemble gives up fitting the training labels exactly in exchange for a boundary that reflects the generating process.

Problem 4: Gradient Boosting from Scratch

We often use OLS because it is simple and interpretable. However, it can suffer from high bias if the true relationship is non-linear. In this problem, you will implement Gradient Boosting manually to correct the bias of an OLS model.

Generate the following non-linear dataset:

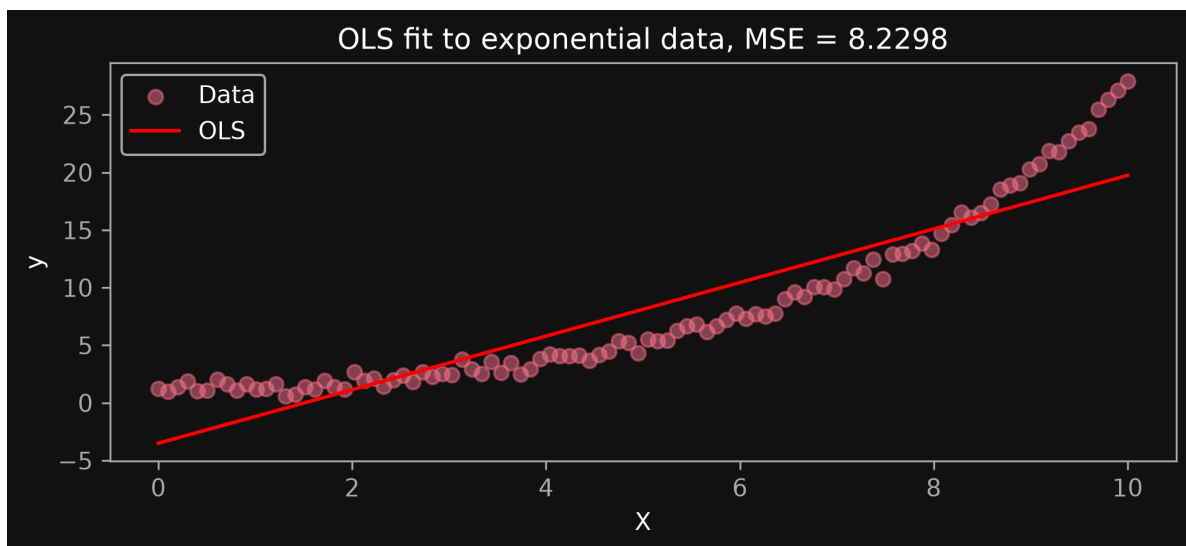
```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.linear_model import LinearRegression
4 from sklearn.tree import DecisionTreeRegressor
5
6 np.random.seed(42)
7 X = np.linspace(0, 10, 100).reshape(-1, 1)
8 # y is exponential, which OLS struggles with
9 y = np.exp(X.ravel() / 3) + np.random.normal(0, 0.5, 100)
```

Problem 4.1: The Base Model (OLS)

- Fit a standard Linear Regression (OLS) model to X and y .
- Calculate and report the Mean Squared Error (MSE).
- Plot the data and the OLS prediction line. Observe how the straight line fails to capture the curve (high bias).

Answer:

MSE = 8.2298.



The line cuts through the curve: it sits below the data at both ends and above it in the middle, so the residuals are a smooth function of X rather than noise. That is what high bias looks like – the error is in the shape of the model, and no amount of extra data would remove it.

Problem 4.2: Boosting with Residuals

Now, we will boost this model by iteratively fitting the residuals with weak learners (shallow trees). Implement the following algorithm:

1. Initialize predictions with the OLS model: $F_0(x) = \hat{y}_{OLS}$.
2. Set learning rate $\eta = 0.1$.
3. Loop 50 times ($m = 1$ to 50):
 - Calculate residuals: $r_m = y - F_{m-1}(x)$.
 - Fit a `DecisionTreeRegressor` with `max_depth=1` to the data (X, r_m) . This is your weak learner $h_m(x)$.
 - Update predictions: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

Answer:

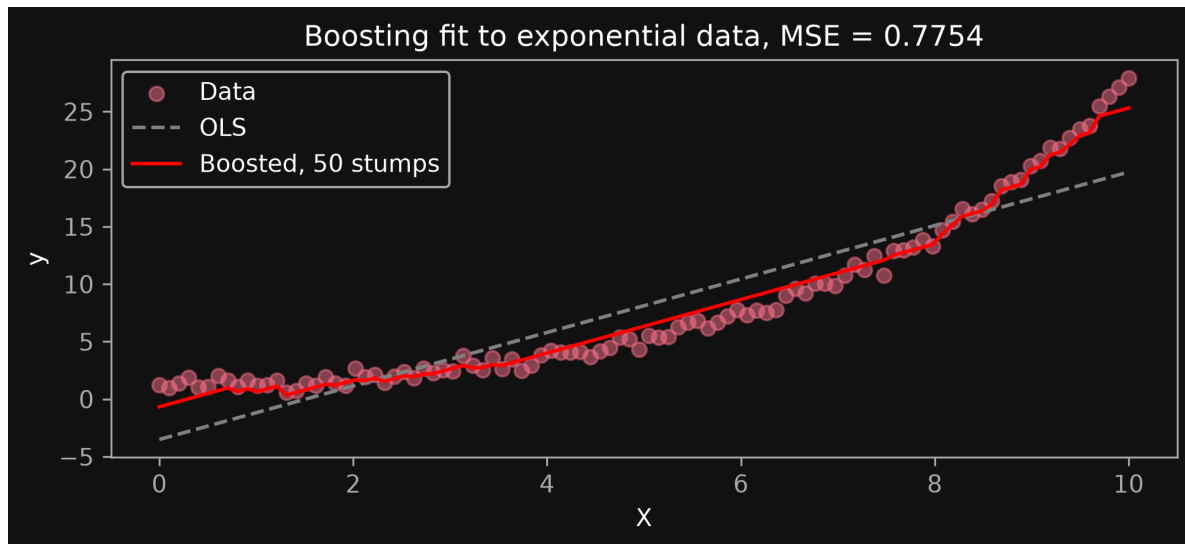
```
1 pred = ols.fittedvalues.copy()
2 eta = 0.1
3 for _ in range(50):
4     resid_m = y - pred
5     tree = DecisionTreeRegressor(max_depth=1, random_state=0).fit(X, resid_m)
6     pred = pred + eta * tree.predict(X)
```

Problem 4.3: Results

- Plot the final boosted prediction $F_{50}(x)$ against the original data.
- Calculate the final MSE. Compare it to the OLS MSE.
- Explain intuitively what the boosting process did to the original linear line.

Answer:

MSE falls from 8.2298 to **0.7754**, a factor of 10.6.



Each stump is one split on X , so it adds one small step to the prediction, placed wherever the current residuals are most lopsided. Fifty of those steps at $\eta = 0.1$ accumulate into a staircase that bends the straight line into the exponential.

The correction is applied where the line was wrong: the steps pile up at the right-hand end, where OLS underfits worst. Nothing here changed the model class – the base learner is still a line, and the trees are fitted to what the line got wrong.

Problem 5: Boosting vs. Random Forest

Using the same dataset as Problem 3 (the checkerboard), we will compare Random Forest (which uses Bagging + feature splitting) and AdaBoost (which reduces bias).

Problem 5.1: Model Comparison

- Fit a `RandomForestClassifier` with 100 estimators.
- Fit an `AdaBoostClassifier` with 50 estimators (base estimator can be a `Tree` with depth 3).
- Report the accuracy of both models on the dataset (since we didn't do a train/test split, reporting training accuracy is fine for this illustrative comparison).

Answer:

	in-sample	held out (70/30)
Random Forest	1.0000	0.8792
AdaBoost	1.0000	0.8389

Both models reach 1.0000 in sample, so as the question warns, that column cannot separate them: 100 unpruned trees and 50 depth-3 trees are both large enough to memorize 496 points.

The held-out column is added because of that. On a 70/30 split (347 train, 149 test) the random forest reaches 0.8792 against 0.8389 for AdaBoost.

Problem 5.2: Conceptual Question

Explain in 2-3 sentences:

- Why does Random Forest help reduce *variance*?
- Why does Boosting help reduce *bias*?

Answer:

Random Forest reduces variance. Each tree is fitted to a bootstrap sample and each split sees a random subset of features, so the trees make different mistakes. Averaging B predictors with correlation ρ leaves variance $\rho\sigma^2 + (1 - \rho)\sigma^2/B$: bagging supplies the $1/B$ term and the random feature subset lowers ρ , which is what the extra randomization buys over plain bagging. It does not touch bias – the average of B trees has the bias of one tree.

Boosting reduces bias. Each round fits the residuals of what is already there, so the ensemble keeps adding structure the current model is missing. Fifty depth-3 stumps can represent shapes no single depth-3 tree can. The cost is the mirror image: errors accumulate rather than average out, so boosting will fit noise if run too long, and it needs a learning rate or an early stop to hold it back.

Problem 6: Clustering from Scratch (DBSCAN Flavor)

Standard K-Means clustering assumes that clusters are spherical and roughly the same size. DBSCAN is an alternative that finds clusters of arbitrary shape. You will implement a simplified version of this.

Generate the “Moons” dataset:

```
1 from sklearn.datasets import make_moons
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 np.random.seed(42)
6 X, _ = make_moons(n_samples=300, noise=0.05)
```

Problem 6.1: The Algorithm

It may be helpful to cache the pairwise distances between points to speed up the algorithm, you can do this using `scipy.spatial.distance.cdist` or numpy operations.

Implement the following simplified DBSCAN logic:

- Set $\epsilon = 0.2$ and $\text{MinPts} = 5$.
- Initialize all points as unvisited.

- Initialize `cluster_id = 0`.
- Loop through each point P in X :
 - If P is visited, continue.
 - Mark P as visited.
 - Find all neighbors of P within distance ϵ .
 - If number of neighbors $< \text{MinPts}$, mark P as Noise (Cluster -1).
 - If number of neighbors $\geq \text{MinPts}$:
 - * Increment `cluster_id`.
 - * Assign P to `cluster_id`.
 - * Expand the cluster (Breadth-First Search): Add all neighbors to a queue. For each point Q in the queue:
 - If Q is unvisited, mark it visited.
 - If Q has enough neighbors ($\geq \text{MinPts}$), add those neighbors to the queue.
 - If Q is not yet assigned to a cluster, assign it to `cluster_id`.

Answer:

```

1 def cdist(a, b):
2     return np.sqrt(((a[:, np.newaxis, :] - b[np.newaxis, :, :]) ** 2).sum(axis=2))
3
4 eps = 0.2
5 min_pts = 5
6 labels = np.full(X_moons.shape[0], -2) # -2: unvisited, -1: noise
7 cluster_id = -1
8 dists = cdist(X_moons, X_moons)
9
10 for i in range(len(X_moons)):
11     if labels[i] != -2:
12         continue
13     neighbors = np.where(dists[i] < eps)[0]
14     if len(neighbors) < min_pts:
15         labels[i] = -1 # noise, but may still be claimed as a border point below
16     else:
17         cluster_id += 1
18         labels[i] = cluster_id
19         queue = list(neighbors)
20         while queue:
21             q = queue.pop(0)
22             if labels[q] >= 0:
23                 continue # already in a cluster
24             labels[q] = cluster_id # unvisited, or previously marked noise
25             q_neighbors = np.where(dists[q] < eps)[0]
26             if len(q_neighbors) >= min_pts:
27                 queue.extend(q_neighbors)

```

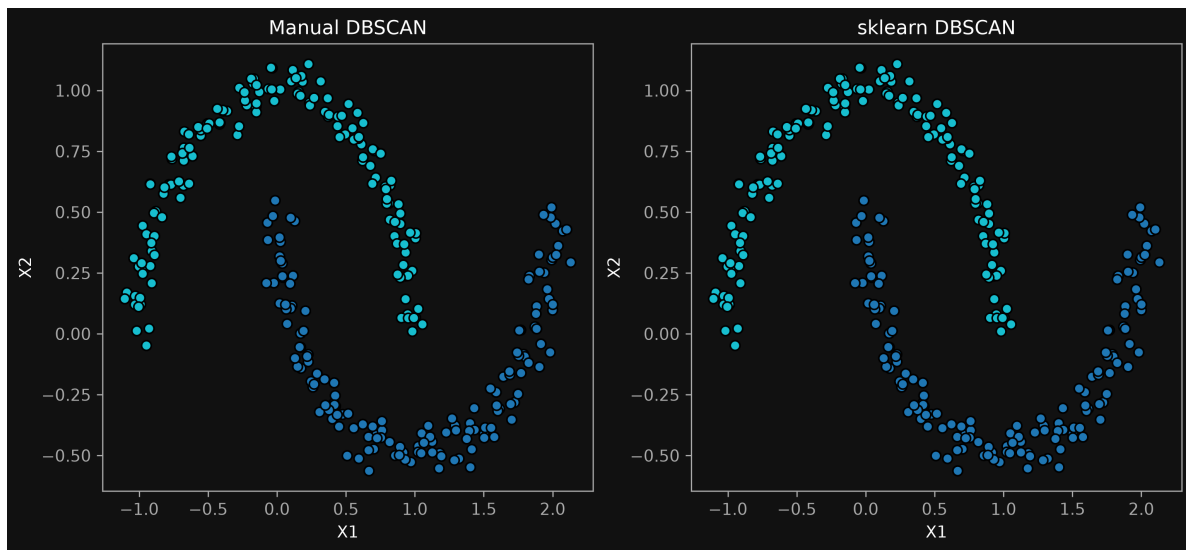
The test inside the queue is `labels[q] >= 0`, not `labels[q] != -2`. A point marked noise earlier can still be a border point of a cluster found later, and that is the behavior `sklearn` implements. Caching the full distance matrix once turns the neighbor lookup into an $O(1)$ row read.

Problem 6.2: Visualization

- Plot the points X , colored by their assigned cluster labels.
- Verify using `sklearn.cluster.DBSCAN` with the same parameters. Do the plots look the same?

Answer:

Identical. Both give two clusters of exactly 150 points and no noise, and `np.array_equal(labels, labels_sklearn)` is `True` – the cluster ids happen to line up too, since both algorithms number clusters in order of first visit.



DBSCAN separates the two moons, which K-means cannot: the moons interlock, so no pair of centroids splits them. DBSCAN never assumes a shape, only that cluster members are reachable through a chain of ϵ -neighbors.